

docker

Prácticas — parte 3: Docker Compose

Antes de empezar

- Puedes usar el entorno en la nube propuesto o tu propio Docker Desktop/VM/WSL2.
- Necesitarás la aplicación que creaste en la Semana 2 (o una nueva, sencilla) para combinarla con una base de datos.
- Hoy hacemos los **Ejercicios 1 a 4** (miércoles) y seguimos con el resto el **viernes**, antes del cuestionario final del bloque.

Ejercicios guiados

Ejercicio 1 — Tu primer docker-compose.yml

1. Crea una carpeta `practica-compose` con un fichero `docker-compose.yml`.
2. Define un único servicio `web` usando la imagen `nginx:1.27` (versión concreta, nunca `latest`) y publicando el puerto 8080 → 80.
3. Levántalo con `docker compose up -d`.
4. Comprueba en el navegador y con `docker compose ps` que funciona.

Resultado esperado: ves la página de nginx en `http://localhost:8080`.

Ejercicio 2 — Añade un segundo servicio

1. En el mismo fichero, añade un servicio `db` con la imagen `postgres:16` y una variable `POSTGRES_PASSWORD`.
2. Ejecuta `docker compose up -d` de nuevo.
3. Comprueba con `docker compose ps` que ahora hay dos servicios en marcha.

Resultado esperado: `docker compose ps` muestra `web` y `db`, ambos "running"/"Up".

Ejercicio 3 — Comunicación por nombre

1. Entra en el contenedor de `web` con `docker compose exec web sh` (o `bash`).
2. Desde dentro, intenta hacer ping o resolver el nombre `db` (si tu imagen no tiene `ping`, instala `curl` / `ping` o usa otra herramienta disponible).
3. Comprueba que `db` se resuelve, aunque no hayas publicado ningún puerto de la base de datos al host.

Resultado esperado: entiendes que los servicios se ven entre sí por nombre, sin necesidad de `ports`.

Ejercicio 4 — Persistencia con volúmenes

1. Añade un volumen con nombre al servicio `db` apuntando a la carpeta de datos de Postgres.
2. Levanta el stack, crea algún dato de prueba dentro de la base de datos (puedes usar `docker compose exec db psql -U postgres`).
3. Haz `docker compose down` (sin `-v`) y vuelve a hacer `docker compose up -d`.
4. Comprueba que el dato de prueba sigue ahí.

Resultado esperado: los datos sobreviven a `docker compose down`.

Ejercicio 5 — depends_on y logs

1. Añade `depends_on` para que `web` (o tu propia app) dependa de `db` .
2. Para todo el stack y vuelve a levantarlo con `docker compose up` (sin `-d` , para ver los logs en directo).
3. Observa el orden en que arrancan los servicios en la salida.
4. En otra terminal, usa `docker compose logs -f db` para seguir solo los logs de la base de datos.

Resultado esperado: puedes explicar, con los logs delante, en qué orden arrancó cada servicio.

Ejercicio 6 — Tu propia app + base de datos

1. Retoma la app de la Semana 2 (o crea una sencilla) y añádela como servicio `app` con `build: .` en el `docker-compose.yml`.
2. Conéctala a `db` usando el nombre del servicio en la cadena de conexión (variable de entorno `environment`).
3. Comprueba que `app` puede leer/escribir en la base de datos.

Resultado esperado: tu aplicación, levantada con `docker compose up -d`, se conecta correctamente a la base de datos del mismo stack.

Retos

Aquí no hay pasos: piensa qué claves y comandos necesitas.

Reto 1 — Variables desde `.env`

Objetivo: mueve la contraseña de la base de datos (y cualquier otro dato sensible) a un fichero `.env`, referenciado desde el `docker-compose.yml` con `${VARIABLE}`, de forma que no quede ninguna contraseña escrita directamente en el YAML.

Reto 2 — Solo lo necesario, accesible

Objetivo: configura tu stack para que la base de datos NO sea accesible desde el navegador/host (sin `ports` para `db`), pero `app` siga funcionando perfectamente contra ella.

Reto 3 — Migración puntual

Objetivo: usando `docker compose run`, ejecuta un comando puntual dentro de un contenedor nuevo del servicio `app` (por ejemplo, un script que imprima la fecha o una migración si tu app la tiene), sin afectar al contenedor de `app` que ya está en marcha.

Reto 4 — Stack completo desde cero

Objetivo: con el stack parado y eliminado por completo (`docker compose down -v`), levanta todo de nuevo con un único `docker compose up -d --build` y demuestra que vuelve a funcionar igual (salvo los datos, que al usar `-v` se habrán perdido a propósito).

Autoevaluación

¿Te ves capaz de...?

- [] Escribir un `docker-compose.yml` con dos o más servicios.
- [] Explicar cómo se comunican dos servicios de la misma Compose sin publicar puertos.
- [] Usar un volumen con nombre para persistir datos de una base de datos.
- [] Explicar qué garantiza (y qué NO garantiza) `depends_on` .
- [] Diferenciar `docker compose up/down` de `stop/start` .
- [] Revisar logs y entrar en un servicio concreto con `exec` .